



Can Large Language Models Improve the Adversarial Robustness of Graph Neural Networks?

Zhongjian Zhang*
zhangzj@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Xiao Wang*
xiao_wang@buaa.edu.cn
Beihang University
Beijing, China

Huichi Zhou
h.zhou24@imperial.ac.uk
Imperial College London
London, UK

Yue Yu
yuyue1218@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Mengmei Zhang
zhangmengmei@bestpay.com.cn
China Telecom Bestpay
Beijing, China

Cheng Yang
yangcheng@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Chuan Shi†
shichuan@bupt.edu.cn
Beijing University of Posts and
Telecommunications
Beijing, China

Abstract

Graph neural networks (GNNs) are vulnerable to adversarial attacks, especially for topology perturbations, and many methods that improve the robustness of GNNs have received considerable attention. Recently, we have witnessed the significant success of large language models (LLMs), leading many to explore the great potential of LLMs on GNNs. However, they mainly focus on improving the performance of GNNs by utilizing LLMs to enhance the node features. Therefore, we ask: *Will the robustness of GNNs also be enhanced with the powerful understanding and inference capabilities of LLMs?* By presenting the empirical results, we find that despite that LLMs can improve the robustness of GNNs, there is still an average decrease of 23.1% in accuracy, implying that the GNNs remain extremely vulnerable against topology attacks. Therefore, another question is *how to extend the capabilities of LLMs on graph adversarial robustness*. In this paper, we propose an LLM-based robust graph structure inference framework, LLM4RGNN¹, which distills the inference capabilities of GPT-4 into a local LLM for identifying malicious edges and an LM-based edge predictor for finding missing important edges, so as to recover a robust graph structure. Extensive experiments demonstrate that LLM4RGNN consistently improves the robustness across various GNNs. Even in some cases where the perturbation ratio increases to 40%, the accuracy of GNNs is still better than that on the clean graph.

*Both authors contributed equally to this research.

†Corresponding author.

¹The source code in <https://github.com/zhongjian-zhang/LLM4RGNN>



This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '25, August 3–7, 2025, Toronto, ON, Canada
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1245-6/25/08
<https://doi.org/10.1145/3690624.3709256>

CCS Concepts

• **Mathematics of computing** → *Graph algorithms*; • **Security and privacy** → *Social network security and privacy*.

Keywords

Graph Neural Networks, Large Language Models, Adversarial Robustness, Structure Learning

ACM Reference Format:

Zhongjian Zhang, Xiao Wang, Huichi Zhou, Yue Yu, Mengmei Zhang, Cheng Yang, and Chuan Shi. 2025. Can Large Language Models Improve the Adversarial Robustness of Graph Neural Networks?. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '25)*, August 3–7, 2025, Toronto, ON, Canada. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3690624.3709256>

1 Introduction

GNNs, as representative graph machine learning methods, effectively utilize their message-passing mechanism to extract useful information and learn high-quality representations from graph data [5, 21, 38, 42, 54]. Despite great success, a host of studies have shown that GNNs are vulnerable to adversarial attacks [19, 25, 32, 36, 43], especially for topology attacks [45, 61, 62], where slightly perturbing the graph structure can lead to a dramatic decrease in the performance. Such vulnerability poses significant challenges for applying GNNs to real-world applications, especially in security-critical scenarios such as finance [41] or medical networks [29].

Threatened by adversarial attacks, several attempts have been made to build robust GNNs, which can be mainly divided into model-centric and data-centric defenses [10, 58]. From the model-centric perspective, defenders can improve robustness through model enhancement, either by robust training schemes [8, 23] or new model architectures [18, 56, 60]. In contrast, data-centric defenses typically focus on flexible data processing to improve the robustness of

GNNs. Treating the attacked topology as noisy, defenders primarily purify graph structures by calculating various similarities between node embeddings [20, 24, 44, 53]. The above methods have received considerable attention in enhancing the robustness of GNNs.

Recently, LLMs such as GPT-4 [1] have demonstrated expressive capabilities in understanding and inferring complex texts, revolutionizing the fields of natural language processing [57, 59], computer vision [48] and graph [28]. The performance of GNNs can be greatly improved by utilizing LLMs to enhance the node features [3, 14, 27]. However, one question remains largely unknown: *Considering the powerful understanding and inference capabilities of LLMs, will LLMs enhance or weaken the adversarial robustness of GNNs to a certain extent?* Answering this question not only helps explore the potential capabilities of LLMs on graphs, but also provides a new perspective on the adversarial robustness of graphs.

Here, we empirically investigate the robustness of GNNs combining six LLMs/LMs (language models), namely OFA-Llama2-7B [27], OFA-SBert [27], TAPE [14], GCN-Llama-7B [3], GCN-e5-large [3], and GCN-SBert [3], against Mettack [62] with a 20% perturbation rate on Cora [30] and PubMed [35] datasets. As shown in Figure 1, the results clearly show that these models suffer from a maximum accuracy decrease of 37.9% and an average of 23.1%, while vanilla GCN [21] experiences a maximum accuracy decrease of 39.1% and an average of 35.5%. It demonstrates that these models remain extremely vulnerable to topology perturbations (more details refer to Section 3). Consequently, another question naturally arises: *How to extend the capabilities of LLMs to improve graph adversarial robustness?* This problem is non-trivial because graph adversarial attacks typically perturb the graph structures, while the capabilities of LLMs usually focus on text processing. Considering that graph structures involve complex interactions among a large number of nodes, how to efficiently explore the inference capabilities of LLMs on perturbed structures is a significant challenge.

In this paper, we propose an LLM-based robust graph structure inference framework, called LLM4RGNN, which efficiently utilizes LLMs to purify the perturbed structure, improving the adversarial robustness. Specifically, based on an open-source and clean graph structure, we design a prompt template that enables GPT-4 [1] to infer how malicious an edge is and provide analysis, to construct an instruction dataset. This dataset is used to fine-tune a local LLM (e.g., Mistral-7B [17] or Llama3-8B [37]), so that the inference capability of GPT-4 can be distilled into the local LLM. When given a new attacked graph structure, we first utilize the local LLM to identify malicious edges. By treating identification results as edge labels, we further distill the inference capability from the local LLM to an LM-based edge predictor, to find missing important edges. Finally, purifying the graph structure by removing malicious edges and adding important edges, makes GNNs more robust. Our contributions can be summarized four-fold:

- To the best of our knowledge, we are the first to explore the potential of LLMs on the graph adversarial robustness. Moreover, we verify the vulnerability of GNNs even with the powerful understanding and inference capabilities of LLMs.
- We propose a novel LLM-based robust graph structure inference framework, called LLM4RGNN, which efficiently utilizes LLMs to make GNNs more robust. Additionally, LLM4RGNN is a general framework, suitable for different LLMs and GNNs.

- Extensive experiments demonstrate that LLM4RGNN consistently improves the robustness of various GNNs against topology attacks. Even in some cases where the perturbation ratio increases to 40%, the accuracy of GNNs with LLM4RGNN is still better than that on the clean graph.
- We utilize GPT-4 to construct an instruction dataset, including GPT-4’s maliciousness assessments and analyses of 26,518 edges. This dataset will be publicly released, which can be used to tune any other LLMs so that they can have the robust graph structure inference capability as GPT-4.

2 Preliminaries

2.1 Text-attributed Graphs (TAGs)

Here, a Text-attributed graph (TAG), defined as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{S})$, is a graph with node-level textual information, where $\mathcal{V} = \{v_1, \dots, v_{|\mathcal{V}|}\}$, $\mathcal{E} = \{e_1, \dots, e_{|\mathcal{E}|}\}$ and $\mathcal{S} = \{s_1, \dots, s_{|\mathcal{V}|}\}$ are the node set, edge set, and text set, respectively. The adjacency matrix of the graph $\mathcal{G}$ is denoted as $\mathbf{A} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$, where $A_{ij} = 1$ if nodes v_i and v_j are connected, otherwise $A_{ij} = 0$. In this work, we focus on the node classification task on TAGs. Specifically, each node v_i corresponds to a label y_i that indicates which category the node v_i belongs to. Usually, we encode the text set $\mathcal{S}$ as the node feature matrix $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_{|\mathcal{V}|}\}$ via some embedding techniques [3, 13, 31] to train GNNs, where $\mathbf{x}_i \in \mathbb{R}^d$. Given some labeled nodes $\mathcal{V}_L \subset \mathcal{V}$, the goal is training a GNN $f(\mathbf{A}, \mathbf{X})$ to predict the labels of the remaining unlabeled nodes $\mathcal{V}_U = \mathcal{V} \setminus \mathcal{V}_L$.

2.2 Graph Adversarial Robustness

This paper primarily focuses on stronger poisoning attacks, which can lead to an extremely low model performance by directly modifying the training data [60, 61]. The formal definition of adversarial robustness against poisoning attacks is as follows:

$$\max_{\delta \in \Delta} \min_{\theta} \mathcal{L}(f_{\theta}(\mathcal{G} + \delta), \mathbf{y}_{\mathcal{T}}), \quad (1)$$

where δ represents a perturbation to the graph $\mathcal{G}$, which may include perturbations to node features, inserting or deleting of edges, etc., Δ represents all permitted and effective perturbations. $\mathbf{y}_{\mathcal{T}}$ is the node labels of the target set $\mathcal{T}$. $\mathcal{L}$ denotes the training loss of GNNs, and θ is the model parameters of f . Equation 1 indicates that under the worst-case perturbation δ , the adversarial robustness of model f is represented by its performance on the target set $\mathcal{T}$. A smaller loss value suggests stronger adversarial robustness, i.e., better model performance. In this paper, we primarily focus on the robustness under two topology attacks: (1) Targeted attacks [61], where attackers aim to mislead the model’s prediction on specific nodes v by manipulating the adjacent edges of v , thus $\mathcal{T} = v$. (2) Non-targeted attacks [43, 62], where attackers aim to degrade the overall performance of GNNs but do not care which node is being targeted, thus $\mathcal{T} = \mathcal{V}_{\text{test}}$, where $\mathcal{V}_{\text{test}}$ denotes the test set.

3 The Adversarial Robustness of GNNs Combining LLMs/LMs

In this section, we empirically investigate whether LLMs enhance or weaken the adversarial robustness of GNNs to a certain extent. Specifically, for the Cora [30] and PubMed [35] datasets, based

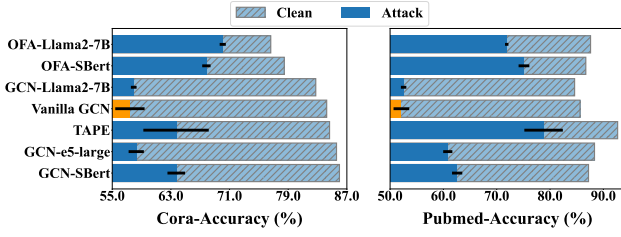


Figure 1: The accuracy of different GNNs combining LLMs/LMs against Mettack with a 20% perturbation rate.

on non-contextualized embeddings encoded by BoW [13] or TF-IDF [34], we employ Mettack [62] with a 20% perturbation rate to generate attack topology. We compare seven representative baselines: TAPE [14] utilizes LLMs to generate extra semantic knowledge relevant to the nodes. OFA [27] employs LLMs to unify different graph data and tasks, where OFA-SBert utilizes Sentence Bert [33] to encode the text of nodes, training and testing the GNNs on each dataset independently. OFA-Llama2-7B involves training a single GNN across the Cora, Pubmed, and OGBN-Arxiv [15] datasets. Following the work [3], GCN-Llama2-7B, GCN-e5-large, and GCN-SBert represent the use of Llama2-7B, e5-large, and Sentence Bert as nodes' text encoders, respectively. The vanilla GCN directly utilizes non-contextual embeddings. By reporting the node classification accuracy on V_{test} to evaluate the robustness of models against the Mettack. As depicted in Figure 1, we observe that under the influence of Mettack, GNNs combining LLMs/LMs suffer from a maximum accuracy decrease of 37.9% and an average decrease of 23.1%, while vanilla GCN [21] suffers from a maximum accuracy decrease of 39.1% and an average accuracy decrease of 35.5%. The results demonstrate that GNNs combining LLMs/LMs remain extremely vulnerable against topology perturbations.

4 LLM4RGNN: The Proposed Framework

In this section, we propose a novel LLM-based robust graph structure inference framework, LLM4RGNN. As shown in Figure 2, LLM4RGNN involves three parts: (a) instruction tuning a local LLM, which distills the inference capability from GPT-4 into a local LLM for identifying malicious edges; (b) training an LM-based edge predictor, which further distills the inference capability from the local LLM into LM-based edge predictor for finding missing important edges; (c) purifying the graph structure by removing malicious edges and adding important edges, making GNNs more robust.

4.1 Instruction Tuning a Local LLM

Given an attacked graph structure, one straightforward method is to query powerful GPT-4 to identify malicious edges on the graph. However, this method is extremely expensive, because there are $|\mathcal{V}|^2$ different perturbation edges on a graph. For example, for the PubMed [35] dataset with 19,717 nodes, the cost is approximately \$9.72 million. Thus, we hope to distill the inference capability of GPT-4 into a local LLM, to identify malicious edges. To this end, instruction tuning based on GPT-4 is a popular fine-tuning technique [4, 47], which utilizes GPT-4 to construct an instruction dataset, and then further trains a local LLM in a supervised fashion. The instruction dataset generally consists of instance (instruction, input, output), where instruction denotes the human instruction

(a task definition in natural language) for LLMs, input is used as supplementary content for the instruction, and output denotes the desired output that follows the instruction. Therefore, the key is how to construct an effective instruction dataset for fine-tuning an LLM to identify the malicious edges.

In the tuning local LLMs phase of Figure 2 (a), based on an open-source and clean graph structure A (TAPE-Arxiv23 [14]), we utilize the existing attacks (Mettack [62], Nettack [61], and Minmax [45]) to generate the perturbed graph structure A' , thus we have the modification matrix S as follows:

$$S = A - \{A'_{\text{Mettack}} \cup A'_{\text{Nettack}} \cup A'_{\text{Minmax}}\}, \quad (2)$$

where $S \in \{-1, 0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ and $S_{ij} = S_{ji} = -1$ when the edge between nodes v_i and v_j is added. Conversely, it is removed if and only if $S_{ij} = S_{ji} = 1$, and $S_{ij} = S_{ji} = 0$ implies that the edge remains unchanged. Here the added edges are considered as negative edge set $\mathcal{E}_n$, i.e., malicious edge set, and the removed edges are considered as positive edge set $\mathcal{E}_p$, i.e., important edge set. Since the attack methods prefer adding edges over removing edges [19], to balance $\mathcal{E}_n$ and $\mathcal{E}_p$, we sample a certain number of clean edges from A to $\mathcal{E}_p$. With $\mathcal{E}_n$ and $\mathcal{E}_p$, we construct the query edge set $\mathcal{E}_q = \mathcal{E}_n \cup \mathcal{E}_p$, which will be used to construct prompts for requesting GPT-4.

Next, based on $\mathcal{E}_q$, we query the GPT-4 in an open-ended manner. This involves prompting the GPT-4 to make predictions on how malicious an edge is and provide analysis for its decisions. With this objective, we design a prompt template that includes "System prompt", which is an open-ended question about how malicious the edge is, and "User content", which is the textual information of node pairs (v_i, v_j) from $\mathcal{E}_p$. The general structure of the template follows: (where "System prompt" and "User content" also respectively correspond to instruction and input in the instruction dataset.)

System prompt: In the context of graph neural networks, attackers manipulate models by adding irrelevant edges or removing relevant ones, leading to incorrect predictions. Your role is crucial in defending against such attacks by evaluating the relevance between pairs of nodes, which will help in identifying and removing the irrelevant edges to mitigate the impact of adversarial attacks on graph-based models. Given textual information about two nodes, analyze the relevance of these two nodes. Provide a concise analysis (approximately 100 words) and assign an integer relevance score from 1 to 6, where 1 indicates completely irrelevant and 6 indicates directly relevant. Your response should be formatted in JSON, with two keys: "Analysis" for your written analysis and "Relevance Score" for your numerical evaluation.

User content: Node $v_i \rightarrow \{\text{Title, Abstract}\}$. \n\n Node $v_j \rightarrow \{\text{Title, Abstract}\}$.

In the "System prompt", we provide background knowledge about tasks and the specific roles played by LLMs in the prompt, which can more effectively harness the inference capability of GPT-4 [14, 49]. Additionally, we require GPT-4 to provide a fine-grained rating of the maliciousness of edges on a scale from 1 to 6, where a lower score indicates more malicious, and a higher score indicates more important. The concept of "Analysis" is particularly crucial, as it not only facilitates an inference process in GPT-4 regarding prediction

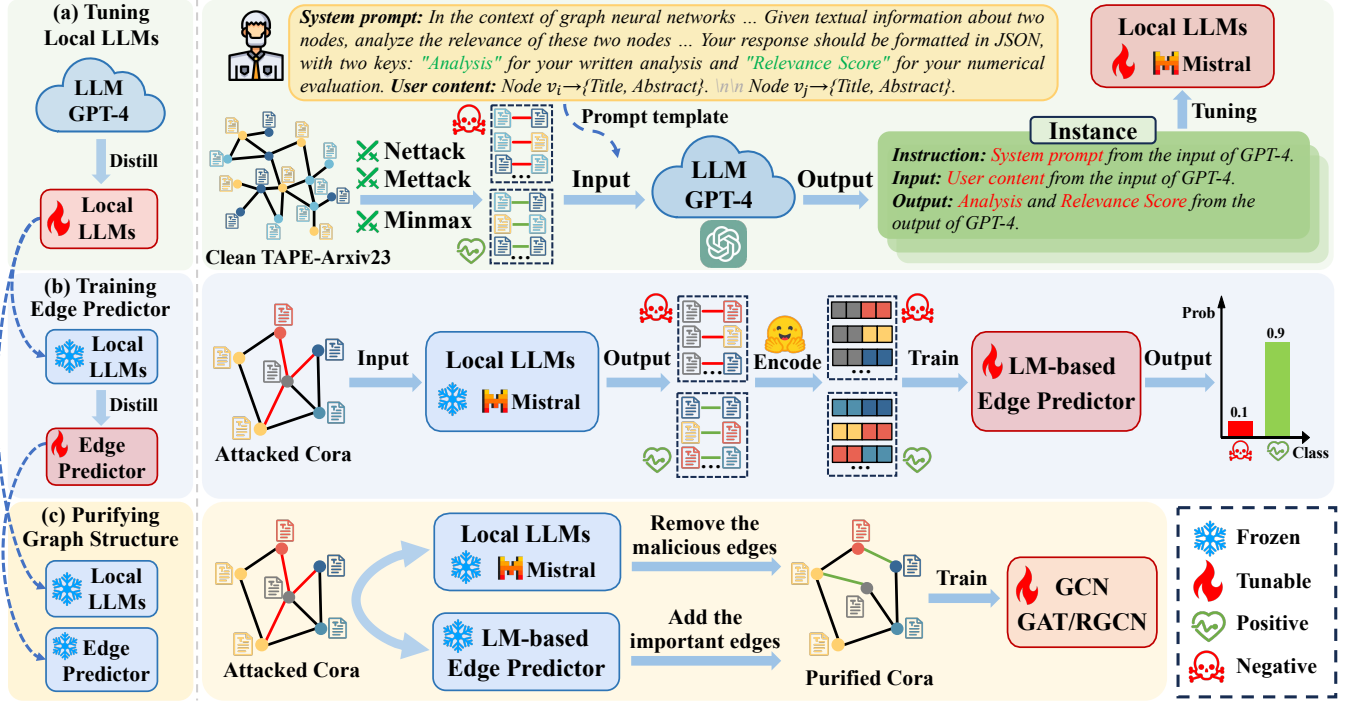


Figure 2: Overall framework of LLM4RGNN.

results, but also serves as a key to distilling the inference capability of GPT-4 into local LLMs. Finally, the output of the instruction dataset is generated by GPT-4 as follows:

Analysis: {Analysis of predicted results}.
Relevance Score: {Predicted integer scores from 1-6}.

In fact, it is difficult for GPT-4 to predict with complete accuracy. To construct a cleaner instruction dataset, we design a post-processing filtering operation. Specifically, for the output of GPT-4, we only preserve the edges with relevance scores $r_e \in \{1, 2, 3\}$ from the negative sample set $\mathcal{E}_n$, and the edges with $r_e \in \{4, 5, 6\}$ from the positive sample set $\mathcal{E}_p$. The refined instruction dataset is then used to fine-tune a local LLM, such as Mistral-7B [17] or Llama3-8B [37]. After that, the well-tuned LLM is able to infer the maliciousness of edges similar to GPT-4. We also provide case studies of GPT-4 and the local LLM (Mistral-7B) in Appendix C.

4.2 Training an LM-based Edge Predictor

Now, given a new attacked graph structure A' , our key idea is to recover a robust graph structure $\hat{A}$. Intuitively, we can input each edge of A' into the local LLM and obtain its relevance score r_e . By removing edges with lower scores, we can mitigate the impact of malicious edges on model predictions. Meanwhile, considering that attackers can also delete some important edges to reduce model performance, we need to find and add important edges that do not exist in A' . Although the local LLM can identify important edges with higher relevance scores, it is still very time and resource-consuming with $|\mathcal{V}|^2$ edges. Therefore, we further design an LM-based edge predictor, as depicted in Figure 2 (b), which utilizes

Sentence Bert [33] as the text encoder and trains a lightweight multilayer perceptron (MLP) to find missing important edges.

Firstly, we introduce how to construct the feature of each edge. Inspired by [3], deep sentence embeddings have emerged as a powerful text encoding method, outperforming non-contextualized embeddings [13, 34]. Furthermore, sentence embedding models offer a lightweight method to obtain representations without fine-tuning. Consequently, for each node v_i , we adopt a sentence embedding model LM as texts encoder to extract representations $\mathbf{h}_i$ from the raw text s_i , i.e., $\mathbf{h}_i = \text{LM}(s_i)$. We concatenate the representations of the node v_i and v_j as the feature for the corresponding edge.

Then the edge label y_e can be derived from r_e as follows:

$$y_e = \begin{cases} 1 & \text{if } r_e > 4 \\ 0 & \text{if } r_e \leq 4 \end{cases}, \quad (3)$$

here, we utilize the local LLM as an edge annotator to distill its inference capability, and select 4 as the threshold to find the most positive edges. It is noted that there may be a label imbalance problem, where the number of positive edges is much higher than the negative. Thus, based on the cosine similarity, we select some node pairs with a lower similarity to construct a candidate set. When there are not enough negative edges, we sample from the candidate set to balance the training set.

Next, we feed the feature of each edge into an MLP to obtain the prediction probability $\hat{y}_e(v_i, v_j) = \text{MLP}(\mathbf{h}_i || \mathbf{h}_j)$. The cross-entropy loss function is used to optimize the parameters of MLP as:

$$\mathcal{L}_{CE}(y_e, \hat{y}_e) = -[y_e \log(\hat{y}_e) + (1 - y_e) \log(1 - \hat{y}_e)]. \quad (4)$$

After training the edge predictor, we input any node pair (v_i, v_j) that does not exist in A' into it to obtain the prediction probability

Table 1: Node classification accuracy ($\% \pm \sigma$) under non-targeted attack (Mettack). Bolded results indicate improved performance.

Dataset Ptb Rate	GCN		GAT		RGCN		SimP-GCN		
	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	
Cora	0%	84.25 $\pm$ 0.36	84.13 $\pm$ 0.33	84.62 $\pm$ 0.47	84.61 $\pm$ 0.39	84.61 $\pm$ 0.46	84.50 $\pm$ 0.46	84.70 $\pm$ 0.62	84.46 $\pm$ 0.67
	5%	75.62 $\pm$ 1.70	81.76$\pm$0.69	79.82 $\pm$ 0.52	81.22$\pm$0.78	76.65 $\pm$ 0.82	82.19$\pm$0.59	79.36 $\pm$ 0.99	81.78$\pm$0.51
	10%	70.72 $\pm$ 2.86	81.80$\pm$0.76	75.33 $\pm$ 1.17	81.86$\pm$0.61	69.62 $\pm$ 1.09	82.14$\pm$0.60	77.00 $\pm$ 1.34	82.19$\pm$0.75
	20%	57.41 $\pm$ 2.00	81.41$\pm$0.77	63.17 $\pm$ 1.82	81.00$\pm$0.98	59.27 $\pm$ 0.81	81.39$\pm$0.44	76.04 $\pm$ 1.66	81.20$\pm$0.63
Citeseer	0%	73.38 $\pm$ 0.72	74.20$\pm$0.56	73.95 $\pm$ 0.45	73.72 $\pm$ 1.07	74.45 $\pm$ 0.31	74.07 $\pm$ 0.90	72.71 $\pm$ 1.14	73.30$\pm$0.62
	5%	69.69 $\pm$ 0.59	73.94$\pm$0.56	71.37 $\pm$ 0.73	73.22$\pm$1.08	72.30 $\pm$ 0.25	73.61$\pm$0.65	71.07 $\pm$ 0.93	73.48$\pm$0.58
	10%	65.75 $\pm$ 0.94	73.62$\pm$0.39	68.40 $\pm$ 1.14	73.21$\pm$1.16	69.36 $\pm$ 0.40	73.41$\pm$0.90	70.55 $\pm$ 0.55	72.84$\pm$0.50
	20%	58.72 $\pm$ 1.00	74.12$\pm$0.85	61.92 $\pm$ 1.76	73.94$\pm$0.67	62.79 $\pm$ 0.60	74.04$\pm$0.70	69.54 $\pm$ 0.78	73.70$\pm$0.64
Pubmed	0%	85.62 $\pm$ 0.10	86.21$\pm$0.13	85.14 $\pm$ 0.11	85.10 $\pm$ 0.09	85.84 $\pm$ 0.10	86.35$\pm$0.10	87.26 $\pm$ 0.08	87.53$\pm$0.14
	5%	73.27 $\pm$ 1.15	85.27$\pm$0.26	80.87 $\pm$ 0.91	84.45$\pm$0.23	81.61 $\pm$ 0.20	85.93$\pm$0.10	85.90 $\pm$ 0.20	86.08$\pm$0.27
	10%	67.54 $\pm$ 0.42	84.95$\pm$0.17	69.94 $\pm$ 4.60	84.88$\pm$0.19	69.55 $\pm$ 0.40	85.79$\pm$0.22	85.71 $\pm$ 0.11	86.06$\pm$0.13
	20%	52.12 $\pm$ 1.47	84.99$\pm$0.32	53.07 $\pm$ 0.69	85.20$\pm$0.30	48.47 $\pm$ 0.71	85.99$\pm$0.21	85.70 $\pm$ 0.15	86.07$\pm$0.24
Arxiv	0%	67.01 $\pm$ 0.08	68.16$\pm$0.36	65.02 $\pm$ 0.37	67.43$\pm$0.49	65.53 $\pm$ 0.38	67.53$\pm$0.38	65.13 $\pm$ 0.31	66.20$\pm$1.16
	5%	50.51 $\pm$ 0.29	68.86$\pm$0.41	54.68 $\pm$ 0.57	68.46$\pm$0.47	52.35 $\pm$ 0.09	68.71$\pm$0.43	51.07 $\pm$ 3.87	67.88$\pm$0.47
	10%	42.91 $\pm$ 0.81	68.65$\pm$0.43	49.18 $\pm$ 0.54	68.56$\pm$0.55	44.75 $\pm$ 0.36	68.54$\pm$0.70	46.52 $\pm$ 6.76	66.47$\pm$2.54
	20%	33.96 $\pm$ 0.46	69.17$\pm$0.43	34.24 $\pm$ 2.12	68.86$\pm$0.54	31.69 $\pm$ 0.63	69.00$\pm$0.59	37.31 $\pm$ 7.59	68.15$\pm$0.65
Products	0%	79.86 $\pm$ 0.15	79.04 $\pm$ 0.42	78.75 $\pm$ 0.26	77.76 $\pm$ 0.62	78.45 $\pm$ 0.25	77.83 $\pm$ 0.48	75.87 $\pm$ 0.36	75.84 $\pm$ 0.29
	5%	66.62 $\pm$ 0.56	76.34$\pm$0.39	76.41 $\pm$ 0.60	76.70$\pm$0.68	71.48 $\pm$ 0.35	75.20$\pm$0.25	64.84 $\pm$ 0.60	72.67$\pm$0.66
	10%	63.31 $\pm$ 0.52	75.80$\pm$0.24	74.13 $\pm$ 0.37	75.48$\pm$0.75	68.98 $\pm$ 0.41	74.75$\pm$0.38	58.59 $\pm$ 1.97	71.80$\pm$0.48
	20%	57.56 $\pm$ 0.64	76.57$\pm$0.46	70.25 $\pm$ 0.82	74.98$\pm$0.53	64.81 $\pm$ 0.31	74.32$\pm$0.38	50.36 $\pm$ 0.96	73.71$\pm$0.81

of edge existence. We have the important edge set for node v_i :

$$\mathcal{E}_{v_i} = \{(v_i, v_j) \mid j \neq i, A'_{ij} = 0, \hat{y}_e > \gamma \text{ and } \hat{y}_e \in \text{Top}_K\}, \quad (5)$$

where $\gamma \in (0, 1)$ is the threshold and K is the maximum number of edges. In this way, we can establish the most important edges for v_i as possible. For all the nodes, we have the final important edge set $\mathcal{E}_{\text{add}} = \bigcup_{v_i \in \mathcal{V}} \mathcal{E}_{v_i}$.

4.3 Purifying Attacked Graph Structure

In Figure 2 (c), the robust graph structure $\hat{\mathbf{A}}$ is derived from the purification of $\mathbf{A}'$. Specifically, new edges from $\mathcal{E}_{\text{add}}$ will be added in $\mathbf{A}'$. Simultaneously, with the relevance score r_e of each edge, we remove the malicious edges in $\mathbf{A}'$ by setting a purification threshold β , i.e., edges with r_e larger than β are preserved, otherwise removed. The $\hat{\mathbf{A}}$ is adaptive to any GNNs, making GNNs more robust.

5 Experiments

5.1 Experimental Setup

Dataset. We conduct experiments on four cross-dataset citation networks (Cora [30], Citeseer [7], Pubmed [35], OGBN-Arxiv [15]) and one cross-domain product network (OGBN-Products [15]). We report the average performance and standard deviation over 10 seeds for each result. More details refer to Appendix A.1.

Baseline. First, LLM4RGNN is a general LLM-based framework to enhance the robustness of GNNs. Therefore, we select the classical GCN [21] and three robust GNNs (GAT [38], RGCN [60] and Simp-GCN [18]) as baselines. Moreover, to more comprehensively evaluate LLM4RGNN, we also compare it with existing SOTA robust GNN frameworks, including ProGNN [20], STABLE [24], HANG-quad [56] and GraphEdit [11], where GCN is selected as the training GNN. Please refer to Appendix A.2 and A.3 for more details.

5.2 Main Result

5.2.1 Against Mettack. Non-targeted attacks aim to disrupt the entire graph topology to degrade the performance of GNNs on the test set. We employ the SOTA non-targeted attack method, Mettack [62], and vary the perturbation rate from 0 to 20% with a step of 5%. We have the following observations: (1) From Table 1, LLM4RGNN consistently improves the robustness across various GNNs. For GCN, there is an average accuracy improvement of 24.3% and a maximum improvement of 103% across five datasets. For robust GNNs, including GAT, RGCN and Simp-GCN, LLM4RGNN on average has 16.6%, 21.4%, and 13.7% relative improvements in accuracy. Notably, despite fine-tuning the local LLM on the TAPE-Arxiv23 dataset, which does not include any medical or product samples, there is still a relative accuracy improvement of 18.8% and 11.4% on the Pubmed and OGBN-Products, respectively. (2) Referring to Table 3, compared with existing robust GNN frameworks, LLM4RGNN achieves SOTA robustness, which benefits from the powerful understanding and inference capabilities of LLMs. (3) Combining Table 1 and Table 3, even in some cases where the perturbation ratio increases to 20%, after using LLM4RGNN to purify the graph structure, the accuracy of GNNs is better than that on the clean graph. One possible reason is that the local LLM effectively identifies malicious edges as negative samples, which helps train a more effective edge predictor to find missing important edges.

5.2.2 Against DICE. To verify the defense generalization capability of LLM4RGNN, we also evaluate its effectiveness against another non-targeted attack, DICE [43]. Notably, DICE is not involved in the construction process of the instruction dataset. Considering that DICE is not as effective as Mettack, we set higher perturbation rates of 10%, 20% and 40%. The results are reported in Table 2 and Table 4.

Table 2: Node classification accuracy ($\% \pm \sigma$) under non-targeted attack (DICE). Bolded results indicate improved performance.

Dataset	Ptb Rate	GCN		GAT		RGCN		SimP-GCN	
		Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN
Cora	10%	81.38 $\pm$ 0.31	81.84$\pm$0.71	81.52 $\pm$ 0.49	82.12$\pm$0.71	81.71 $\pm$ 0.52	82.14$\pm$0.52	82.29 $\pm$ 0.55	82.35$\pm$0.65
	20%	78.38 $\pm$ 0.51	80.96$\pm$0.61	78.51 $\pm$ 0.35	81.02$\pm$1.00	78.58 $\pm$ 0.55	81.52$\pm$0.88	79.65 $\pm$ 0.61	81.38$\pm$0.56
	40%	71.62 $\pm$ 0.40	79.46$\pm$0.93	71.81 $\pm$ 0.72	79.11$\pm$1.00	72.77 $\pm$ 0.48	79.86$\pm$0.86	74.37 $\pm$ 0.67	79.64$\pm$1.14
Citeseer	10%	71.08 $\pm$ 0.63	73.95$\pm$0.59	71.62 $\pm$ 0.79	73.73$\pm$0.58	72.74 $\pm$ 0.46	73.85$\pm$1.07	71.15 $\pm$ 0.75	73.41$\pm$0.64
	20%	68.79 $\pm$ 0.49	73.32$\pm$0.58	69.68 $\pm$ 0.59	73.26$\pm$0.45	70.67 $\pm$ 0.33	73.61$\pm$0.69	70.67 $\pm$ 0.53	72.66$\pm$0.73
	40%	63.98 $\pm$ 1.02	73.35$\pm$0.50	64.81 $\pm$ 0.87	73.07$\pm$0.71	65.46 $\pm$ 0.55	73.64$\pm$1.02	68.53 $\pm$ 1.80	72.80$\pm$0.63
Pubmed	10%	82.86 $\pm$ 0.17	85.14$\pm$0.22	81.85 $\pm$ 0.17	83.96$\pm$0.22	83.48 $\pm$ 0.12	85.24$\pm$0.15	86.60 $\pm$ 0.07	86.92$\pm$0.13
	20%	80.11 $\pm$ 0.16	84.96$\pm$0.19	78.92 $\pm$ 0.20	84.33$\pm$0.28	80.55 $\pm$ 0.11	85.06$\pm$0.20	85.98 $\pm$ 0.08	86.66$\pm$0.18
	40%	74.45 $\pm$ 0.19	84.89$\pm$0.21	72.47 $\pm$ 0.24	83.98$\pm$0.17	74.78 $\pm$ 0.18	84.73$\pm$0.25	85.36 $\pm$ 0.09	86.42$\pm$0.11
Arxiv	10%	64.23 $\pm$ 0.10	68.15$\pm$0.89	62.39 $\pm$ 0.28	67.96$\pm$1.05	62.92 $\pm$ 0.27	67.99$\pm$1.00	62.73 $\pm$ 0.27	66.77$\pm$1.51
	20%	62.35 $\pm$ 0.17	68.21$\pm$1.01	59.60 $\pm$ 0.40	67.84$\pm$1.10	60.58 $\pm$ 0.27	67.95$\pm$1.11	60.52 $\pm$ 0.25	66.56$\pm$1.19
	40%	57.55 $\pm$ 0.14	68.73$\pm$0.32	54.86 $\pm$ 0.38	68.52$\pm$0.53	56.33 $\pm$ 0.21	68.47$\pm$0.40	56.38 $\pm$ 0.23	67.37$\pm$1.64
Products	10%	76.05 $\pm$ 0.16	77.66$\pm$0.22	75.12 $\pm$ 0.28	76.16$\pm$0.31	74.78 $\pm$ 0.33	76.41$\pm$0.43	71.62 $\pm$ 0.61	74.20$\pm$0.41
	20%	72.48 $\pm$ 0.21	77.15$\pm$0.62	72.03 $\pm$ 0.37	75.62$\pm$1.06	71.71 $\pm$ 0.27	76.12$\pm$0.59	68.03 $\pm$ 0.44	74.57$\pm$1.00
	40%	66.26 $\pm$ 0.37	77.02$\pm$0.56	65.52 $\pm$ 0.62	76.20$\pm$0.40	64.70 $\pm$ 0.31	75.85$\pm$0.46	60.41 $\pm$ 0.57	75.01$\pm$0.64

Table 3: Node classification accuracy ($\% \pm \sigma$) under non-targeted attack (Mettack). The best results are in bold. OOT means that the result could not be obtained in 15 days.

Dataset	Ptb Rate	Pro-GNN	STABLE	HANG-quad	GraphEdit	LLM4RGNN
Cora	0%	80.85 $\pm$ 0.44	85.09$\pm$0.21	79.41 $\pm$ 0.55	76.35 $\pm$ 0.38	84.13 $\pm$ 0.33
	5%	79.81 $\pm$ 0.45	80.53 $\pm$ 1.13	78.81 $\pm$ 1.02	74.97 $\pm$ 0.95	81.76$\pm$0.69
	10%	78.57 $\pm$ 0.97	79.53 $\pm$ 0.52	78.15 $\pm$ 1.14	74.87 $\pm$ 0.75	81.80$\pm$0.76
	20%	76.07 $\pm$ 0.57	78.70 $\pm$ 0.89	74.90 $\pm$ 0.65	73.82 $\pm$ 0.48	81.41$\pm$0.77
Citeseer	0%	71.11 $\pm$ 0.45	72.51 $\pm$ 1.37	71.18 $\pm$ 0.64	72.60 $\pm$ 0.57	74.20$\pm$0.56
	5%	69.68 $\pm$ 0.53	71.23 $\pm$ 1.35	71.16 $\pm$ 0.71	71.40 $\pm$ 0.72	73.94$\pm$0.56
	10%	68.73 $\pm$ 0.79	70.47 $\pm$ 1.47	70.84 $\pm$ 1.00	71.35 $\pm$ 0.98	73.62$\pm$0.39
	20%	68.37 $\pm$ 0.84	67.91 $\pm$ 1.98	70.00 $\pm$ 1.11	69.44 $\pm$ 0.86	74.12$\pm$0.85
Pubmed	0%	OOT	84.07 $\pm$ 0.17	84.98 $\pm$ 0.13	85.38 $\pm$ 0.07	86.21$\pm$0.13
	5%	OOT	79.41 $\pm$ 0.86	84.97 $\pm$ 0.16	84.77 $\pm$ 0.09	85.27$\pm$0.26
	10%	OOT	77.65 $\pm$ 0.25	84.88 $\pm$ 0.19	83.41 $\pm$ 0.19	84.95$\pm$0.17
	20%	OOT	72.51 $\pm$ 1.05	84.94 $\pm$ 0.19	82.34 $\pm$ 0.27	84.99$\pm$0.32
Arxiv	0%	OOT	66.70 $\pm$ 0.19	67.85 $\pm$ 0.17	-	68.16$\pm$0.36
	5%	OOT	61.40 $\pm$ 0.50	63.53 $\pm$ 0.42	-	68.86$\pm$0.41
	10%	OOT	59.37 $\pm$ 0.40	57.49 $\pm$ 0.76	-	68.65$\pm$0.43
	20%	OOT	58.24 $\pm$ 0.60	41.61 $\pm$ 1.28	-	69.17$\pm$0.43
Products	0%	OOT	79.23 $\pm$ 0.33	80.61$\pm$0.18	-	79.04 $\pm$ 0.42
	5%	OOT	76.24 $\pm$ 0.44	72.41 $\pm$ 0.66	-	76.34$\pm$0.39
	10%	OOT	74.38 $\pm$ 0.40	69.88 $\pm$ 0.86	-	75.80$\pm$0.24
	20%	OOT	72.21 $\pm$ 0.40	62.24 $\pm$ 0.28	-	76.57$\pm$0.46

Similar to the results under Mettack, LLM4RGNN consistently improves the robustness across various GNNs and is superior to other robust GNN frameworks. For GCN, GAT, RGCN and SimP-GCN, LLM4RGNN on average brings 8.2%, 8.8%, 8.1% and 6.5% relative improvements in accuracy on five datasets. Remarkably, even in some cases where the perturbation ratio increases to 40%, the accuracy of GNNs is better than that on the clean graph.

5.2.3 Against Nettack. Unlike non-targeted attacks, targeted attacks focus on fooling GNNs into misclassifying a particular node v . Here, we employ the SOTA targeted attack, Nettack [61]. Following previous work [60], nodes with a degree greater than 10 are selected as the target nodes, and the number of perturbations applied to the targeted node from 0 to 5 with a step of 1. As shown in Figure 3 and

Table 4: Node classification accuracy ($\% \pm \sigma$) under non-targeted attack (DICE). The best results are in bold. OOT means that the result could not be obtained in 15 days.

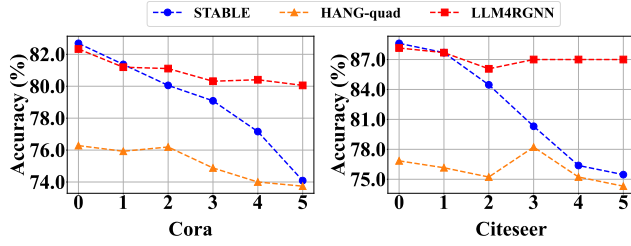
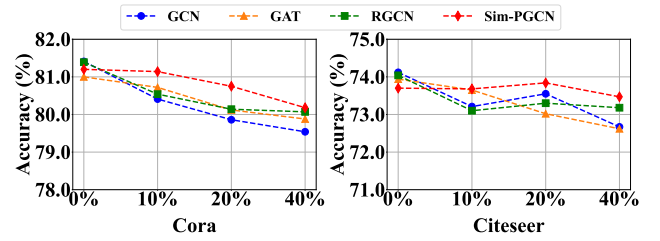
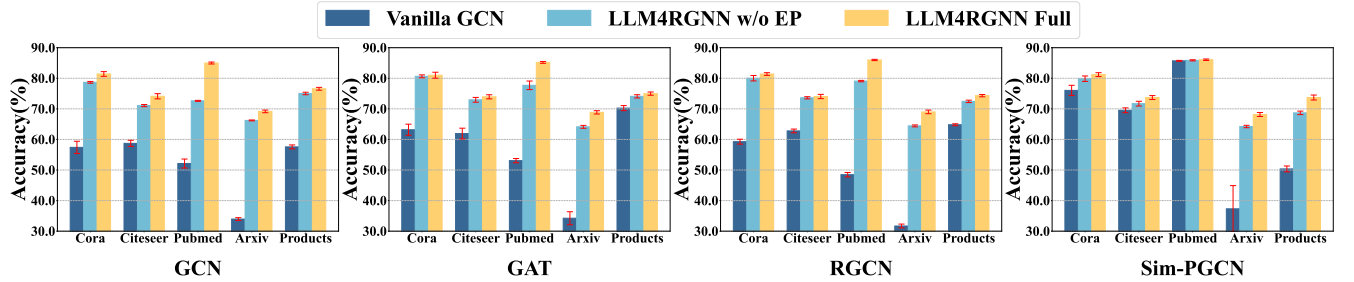
Dataset	Ptb Rate	Pro-GNN	STABLE	HANG-quad	GraphEdit	LLM4RGNN
Cora	0%	80.85 $\pm$ 0.44	85.09$\pm$0.21	79.41 $\pm$ 0.55	76.35 $\pm$ 0.38	84.13 $\pm$ 0.33
	10%	81.56 $\pm$ 0.36	81.33 $\pm$ 0.72	78.26 $\pm$ 0.29	73.73 $\pm$ 0.57	81.84$\pm$0.71
	20%	78.32 $\pm$ 0.33	78.80 $\pm$ 0.83	76.45 $\pm$ 0.50	69.86 $\pm$ 0.71	80.96$\pm$0.61
	40%	71.76 $\pm$ 0.34	76.72 $\pm$ 0.93	74.28 $\pm$ 0.42	66.95 $\pm$ 0.48	79.46$\pm$0.93
Citeseer	0%	71.11 $\pm$ 0.45	72.51 $\pm$ 1.37	71.18 $\pm$ 0.64	72.60 $\pm$ 0.57	74.20$\pm$0.56
	10%	70.81 $\pm$ 0.68	70.27 $\pm$ 1.62	70.29 $\pm$ 0.70	68.99 $\pm$ 0.52	73.95$\pm$0.59
	20%	69.21 $\pm$ 0.79	69.11 $\pm$ 1.57	70.05 $\pm$ 0.77	69.31 $\pm$ 0.41	73.32$\pm$0.58
	40%	68.04 $\pm$ 1.38	67.55 $\pm$ 0.86	68.79 $\pm$ 1.09	66.59 $\pm$ 0.74	73.35$\pm$0.50
Pubmed	0%	OOT	84.07 $\pm$ 0.17	84.98 $\pm$ 0.13	85.38 $\pm$ 0.07	86.21$\pm$0.13
	10%	OOT	81.68 $\pm$ 0.21	84.97 $\pm$ 0.15	82.12 $\pm$ 0.07	85.14$\pm$0.22
	20%	OOT	79.08 $\pm$ 0.23	84.94 $\pm$ 0.18	81.11 $\pm$ 0.10	84.96$\pm$0.19
	40%	OOT	74.51 $\pm$ 0.45	84.86 $\pm$ 0.20	83.04 $\pm$ 0.09	84.89$\pm$0.21
Arxiv	0%	OOT	66.70 $\pm$ 0.19	67.85 $\pm$ 0.17	-	68.16$\pm$0.36
	10%	OOT	64.93 $\pm$ 0.19	65.99 $\pm$ 0.19	-	68.15$\pm$0.89
	20%	OOT	62.64 $\pm$ 0.44	63.74 $\pm$ 0.23	-	68.21$\pm$1.01
	40%	OOT	58.55 $\pm$ 0.37	59.23 $\pm$ 0.17	-	68.73$\pm$0.32
Products	0%	OOT	79.23 $\pm$ 0.33	80.61$\pm$0.18	-	79.04 $\pm$ 0.42
	10%	OOT	76.72 $\pm$ 0.32	77.06 $\pm$ 0.29	-	77.66$\pm$0.22
	20%	OOT	75.15 $\pm$ 0.27	75.52 $\pm$ 0.24	-	77.15$\pm$0.62
	40%	OOT	73.68 $\pm$ 0.26	69.01 $\pm$ 0.17	-	77.02$\pm$0.56

Table 5, results indicate that LLM4RGNN not only consistently improves the robustness of various GNNs, but also surpasses existing robust frameworks, exhibiting exceptional resistance to Nettack.

5.2.4 Against Adaptive Attack. Consider the worst-case scenario, where attackers have access to the well-tuned Mistral-7B. Thus, attackers can adaptively generate malicious edges that Mistral-7B rating ≥ 4 , to avoid being removed. Based on Pubmed with more edges, we control the perturbation ratios at 5% and 10%. As reported in Tabel 6, the results indicate that LLM4RGNN effectively defends against adaptive attack and consistently improves the robustness of various GNNs. Although malicious edges $r_e \in \{4, 5, 6\}$ failed to be removed, they are less aggressive than those $r_e \in \{1, 2, 3\}$, and the important edges can further mitigate their impact.

Table 5: Node classification accuracy ($\% \pm \sigma$) under targeted attack (Nettack). Bolded results indicate improved performance.

Dataset	Ptb Num	GCN		GAT		RGCN		Sim-PGCN	
		Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN
Cora	0	85.92 $\pm$ 2.21	87.50 $\pm$ 1.79	85.66 $\pm$ 1.90	87.11 $\pm$ 1.64	86.45 $\pm$ 1.87	88.55 $\pm$ 2.57	86.97 $\pm$ 1.09	86.71 $\pm$ 2.39
	1	79.87 $\pm$ 2.50	85.79 $\pm$ 1.75	82.11 $\pm$ 2.06	86.71 $\pm$ 2.24	82.50 $\pm$ 1.87	87.37 $\pm$ 2.64	81.71 $\pm$ 2.39	86.05 $\pm$ 2.37
	2	76.32 $\pm$ 1.77	85.66 $\pm$ 2.08	75.13 $\pm$ 4.42	86.18 $\pm$ 1.79	77.37 $\pm$ 1.29	87.37 $\pm$ 1.58	77.76 $\pm$ 1.37	84.34 $\pm$ 1.99
	3	71.45 $\pm$ 2.43	84.47 $\pm$ 1.15	70.79 $\pm$ 3.85	85.00 $\pm$ 2.58	72.11 $\pm$ 1.64	86.32 $\pm$ 2.44	71.97 $\pm$ 2.89	85.39 $\pm$ 1.37
	4	64.47 $\pm$ 3.38	84.61 $\pm$ 2.04	65.79 $\pm$ 3.86	85.39 $\pm$ 1.99	67.24 $\pm$ 1.90	86.84 $\pm$ 1.95	63.55 $\pm$ 3.58	84.61 $\pm$ 2.50
	5	55.13 $\pm$ 4.01	84.08 $\pm$ 1.61	60.00 $\pm$ 4.68	83.03 $\pm$ 1.90	60.26 $\pm$ 2.48	85.13 $\pm$ 1.56	52.63 $\pm$ 4.40	83.55 $\pm$ 1.69
Citeseer	0	86.31 $\pm$ 1.28	86.77 $\pm$ 1.02	86.92 $\pm$ 0.77	87.38 $\pm$ 1.02	85.23 $\pm$ 1.41	86.92 $\pm$ 1.72	80.92 $\pm$ 2.50	84.62 $\pm$ 2.84
	1	84.62 $\pm$ 2.57	86.46 $\pm$ 0.62	85.54 $\pm$ 1.23	86.92 $\pm$ 1.03	82.15 $\pm$ 1.85	86.31 $\pm$ 1.45	80.00 $\pm$ 3.30	84.31 $\pm$ 2.36
	2	81.69 $\pm$ 2.62	85.38 $\pm$ 1.03	83.38 $\pm$ 1.79	86.38 $\pm$ 1.61	74.46 $\pm$ 4.25	85.85 $\pm$ 1.51	79.54 $\pm$ 4.30	84.62 $\pm$ 1.54
	3	79.08 $\pm$ 3.24	86.00 $\pm$ 0.63	81.38 $\pm$ 3.11	85.38 $\pm$ 1.42	72.00 $\pm$ 3.94	85.54 $\pm$ 1.41	76.62 $\pm$ 5.99	83.85 $\pm$ 2.31
	4	76.77 $\pm$ 5.26	86.00 $\pm$ 0.83	77.08 $\pm$ 4.54	85.69 $\pm$ 1.55	69.08 $\pm$ 2.70	85.38 $\pm$ 1.58	73.23 $\pm$ 7.51	84.00 $\pm$ 1.41
	5	74.31 $\pm$ 5.29	86.00 $\pm$ 1.08	73.69 $\pm$ 8.44	85.54 $\pm$ 1.02	64.31 $\pm$ 3.94	84.46 $\pm$ 1.45	68.31 $\pm$ 10.62	82.46 $\pm$ 2.20

**Figure 3: Node classification accuracy under targeted attack (Nettack) with different perturbations.****Figure 4: Node classification accuracy of LLM4RGNN under Mettack-20% with different text perturbations.****Figure 5: Performance comparison in different settings against Mettack with a 20% perturbation rate.****Table 6: Node classification accuracy of Pubmed under adaptive attack. Bolded results indicate improved performance.**

Ptb Rate	GCN		RGCN		SimP-GCN	
	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN
0%	85.62 $\pm$ 0.10	86.21 $\pm$ 0.13	85.84 $\pm$ 0.10	86.35 $\pm$ 0.10	87.26 $\pm$ 0.08	87.53 $\pm$ 0.14
5%	84.23 $\pm$ 0.12	84.70 $\pm$ 0.28	84.49 $\pm$ 0.10	85.43 $\pm$ 0.20	86.21 $\pm$ 0.10	86.36 $\pm$ 0.25
10%	81.51 $\pm$ 0.20	84.82 $\pm$ 0.26	82.69 $\pm$ 0.21	85.58 $\pm$ 0.20	85.86 $\pm$ 0.16	86.11 $\pm$ 0.16

5.3 Model Analysis

5.3.1 Ablation Study. We conduct ablation studies under Mettack with a 20% perturbation rate, where GCN is selected as training GNN. The results are depicted in Figure 5, where "Vanilla" is without any modifications to the attacked topology. The "w/o EP" only removes malicious edges by the local LLM, while "Full" includes both removing malicious edges and adding important edges. Across all settings, LLM4RGNN Full consistently outperforms other settings. Using the local LLM to remove most malicious edges can significantly reduce the impact of adversarial perturbation, improving the accuracy of GNNs. By using the edge predictor to add important

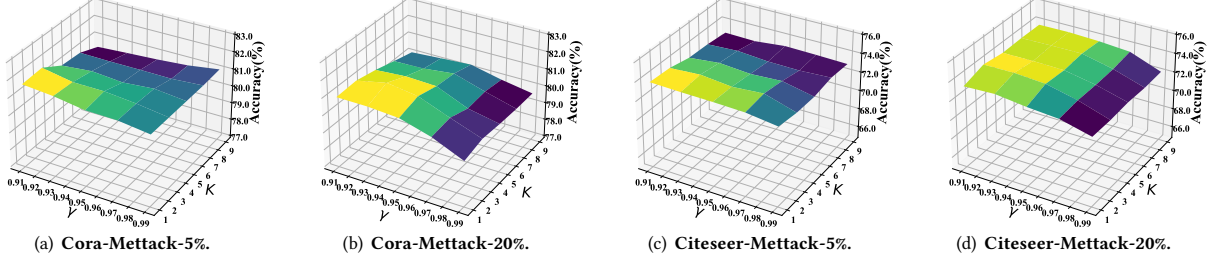
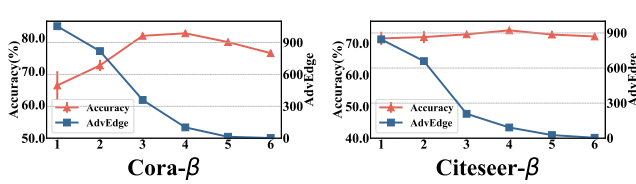
neighbors for each node, additional information gain is provided to the center nodes, further improving the accuracy of GNNs.

5.3.2 Impact of Text Quality. Considering that LLM4RGNN relies on textual information for reasoning, we analyze the impact of text quality on the model's effectiveness. Specifically, against the worst-case scenario of 20% Mettack, we further add random text replacement perturbations to the Cora and Citeseer to reduce the text quality of nodes. As shown in Figure 4, results show that LLM4RGNN's performance varies by less than two points under 10-40% perturbations, and surpasses existing robust GNN frameworks. A possible explanation is that LLMs have strong robustness to text perturbations [2], and LLM4RGNN fully inherits it.

5.3.3 Comparison with Different LLMs. To test LLM4RGNN's generalization across different LLMs, we choose four popular open source LLMs, Llama2-7B/13B, Llama3-8B and Mistral-7B, as the starting checkpoints of LLM. We also introduce a direct comparison

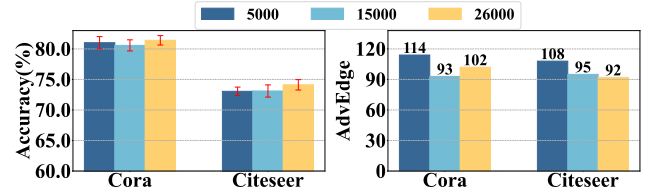
Table 7: Performance comparison with different LLMs against Mettack with a 20% perturbation rate.

LLM		Cora			Citeseer		
		AdvEdge ($\downarrow$)	ACC ($\uparrow$) w/o EP	ACC ($\uparrow$) Full	AdvEdge ($\downarrow$)	ACC ($\uparrow$) w/o EP	ACC ($\uparrow$) Full
Close source	GPT-3.5	186(3.52%)	73.56 $\pm$ 0.49	75.36 $\pm$ 1.73	231(5.47%)	66.69 $\pm$ 1.04	68.92 $\pm$ 1.63
	GPT-4	90(1.71%)	79.83$\pm$0.31	81.76$\pm$0.87	73(1.73%)	71.04$\pm$0.57	72.81$\pm$0.60
Open source	Llama2-7B	156(2.96%)	78.76 $\pm$ 0.42	80.99 $\pm$ 0.78	203(4.80%)	69.83 $\pm$ 0.78	72.86 $\pm$ 0.93
	Llama2-13B	132(2.50%)	79.00 $\pm$ 0.30	81.35 $\pm$ 1.15	93(2.20%)	70.81 $\pm$ 0.73	72.84 $\pm$ 0.85
	Llama3-8B	107(2.03%)	79.22 $\pm$ 0.34	81.71$\pm$0.51	91(2.15%)	70.85 $\pm$ 0.64	74.06 $\pm$ 0.78
	Mistral-7B	102(1.93%)	78.68 $\pm$ 0.28	81.41 $\pm$ 0.77	92(2.18%)	71.07$\pm$0.35	74.12$\pm$0.85

**Figure 6: Analysis of the hyper-parameter γ and K against Mettack.****Figure 7: Analysis of hyper-parameter β against Mettack-20%.**

using the closed source GPT-3.5 and GPT-4. The metric AdvEdge ($\downarrow$) is introduced to measure the number and proportion of malicious edges remaining after the LLM performs filtering. We report the results of GCN on Cora and Citeseer under Mettack with a perturbation ratio of 20% (generating 1053 malicious edges for Cora and 845 for Citeseer). As reported in Table 7, we observe: (1) The well-tuned local LLMs are significantly superior to GPT-3.5 in identifying malicious edges, and the trained edge predictor consistently improves accuracy, indicating that the inference capability of GPT-4 is effectively distilled into LLMs and edge predictors. (2) A stronger LLM yields better overall performance. Among them, the performance of Mistral-7B and Llama3-8B is comparable to that of GPT-4. We provide a detailed comparison between them in Appendix B.4.

5.3.4 Hyper-parameter Analysis. First, we present the accuracy of LLM4RGNN under different combinations of the probability threshold λ and the maximum number of important edges K in Figure 6. The results indicate that the accuracy of LLM4RGNN varies minimally across different hyper-parameter settings, demonstrating its insensitivity to the hyper-parameters λ and K . More results refer to Appendix B.3. Besides, we report the accuracy and AdvEdge of LLM4RGNN under different β , the purification threshold of whether an edge is preserved or not. As shown in Figure 7, when the β is set to 4, most malicious edges are identified and achieve optimal performance. This is because a low β fails to identify malicious edges effectively, while a high β removes more malicious edges but could remove some useful edges. Lastly, we set the number of instances

**Figure 8: Analysis of instances number against Mettack-20%.**

for tuning the local LLM to 5000, 15000, and 26000, respectively, to analyze the model’s effectiveness with different instance counts. As shown in Figure 8, with only 5000 instances, the tuned local LLM can effectively identify malicious edges, surpassing the existing SOTA method, which indicates that the excellent robustness of LLM4RGNN can be achieved on a lower budget, approximately \$46.7. Detailed tokens cost refer to Appendix A.4.

Table 8: Average inference times (s) for different datasets.

Component	Cora	Citeseer	Products	Pubmed	Arxiv
Local LLM	0.54	0.57	0.63	0.64	1.12
Edge Predictor	0.05	0.06	0.24	0.31	0.25

5.3.5 Efficiency Analysis. In LLM4RGNN, using 26,518 samples to fine-tune the local LLM is a one-time process, controlled within 9 hours. The edge predictor is only a lightweight MLP, with training time on each dataset controlled within 3 minutes. The complexity of LLM inferring edge relationships is $O(|\mathcal{E}|)$, and the edge predictor is $O(|\mathcal{V}|^2)$, and their inference processes are parallelizable. We provide the average time for LLM to infer one edge and for the lightweight edge predictor to infer one node in Table 8. Overall, the average inference time for the local LLM and the edge predictor is 0.7s and 0.2s, respectively, which is acceptable. Each experiment’s total inference time is controlled within 90 minutes. Here, we further discuss how to extend LLM4RGNN to large-scale graphs. For the local LLM with complexity $O(|\mathcal{E}|)$, we introduce the parallel inference framework vLLM [22] and cache the edges inferred by the LLM. For the edge predictor with complexity $O(|\mathcal{V}|^2)$, we infer

Table 9: Statistics of edges purified by LLM4RGNN. We also report the percentage of malicious edges removed.

Dataset	Type	Mettack			DICE		
		5%	10%	20%	10%	20%	40%
Cora	Remove	234(89.3%)	466(88.6%)	951(90.3%)	216(87.8%)	429(86.1%)	932(88.9%)
	Add	9,362	6,440	9,878	6,307	5,928	8,495
Citeseer	Remove	181(85.8%)	367(87.0%)	753(89.1%)	184(91.1%)	365(93.6%)	753(91.7%)
	Add	12,464	14,332	7,215	10,569	12,434	11,372
Pubmed	Remove	1,401(63.2%)	2,708(61.1%)	4,972(56.9%)	1,188(53.7%)	2,281(51.3%)	4,749(53.6%)
	Add	56,505	100,122	97,057	57,595	96,617	91,525
Arxiv	Remove	799(95.5%)	1,600(95.6%)	3,204(96.3%)	786(96.7%)	1,629(96.3%)	3,227(95.9%)
	Add	49,741	52,253	52,621	53,190	47,046	46,807
Products	Remove	483(65.2%)	1,001(67.5%)	2,059(69.5%)	679(94.8%)	1,400(94.2%)	2,799(94.4%)
	Add	25,413	22,731	46,827	24,177	44,041	96,945

Table 10: Performance of complete OGBN-Arxiv under DICE. Bolded results indicate improved performance.

Ptb Rate	GCN		GAT		SimP-GCN	
	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN	Vanilla	LLM4RGNN
0%	68.33 $\pm$ 0.04	68.82$\pm$0.14	64.99 $\pm$ 0.10	66.48$\pm$0.16	64.50 $\pm$ 3.51	65.54$\pm$3.94
10%	65.22 $\pm$ 0.05	69.26$\pm$0.04	61.14 $\pm$ 0.08	67.85$\pm$0.11	61.41 $\pm$ 4.76	67.25$\pm$2.92
20%	62.13 $\pm$ 0.09	68.80$\pm$0.07	57.77 $\pm$ 0.13	67.46$\pm$0.08	59.39 $\pm$ 4.68	66.74$\pm$3.16
40%	56.75 $\pm$ 0.05	68.40$\pm$0.05	52.04 $\pm$ 0.11	66.78$\pm$0.09	55.65 $\pm$ 2.09	66.26$\pm$2.90

only the top K most similar nodes for each node (where $K \ll |\mathcal{V}|$), to reduce the complexity to $O(K|\mathcal{V}|)$. As reported in Table 10, results on complete OGBN-Arxiv (169,343 nodes and 1,166,243 edges) indicate the effectiveness of LLM4RGNN on the large-scale graph, with an average inference time of 7 hours (where K is set to 2000).

5.3.6 Purifying Edge Statistic. To analyze how LLM4RGNN defends various attacks, we reported the number of edges added and removed, and the percentage of malicious edges removed in Table 9. We find that LLM4RGNN removes an average of 82.4% of malicious edges across five datasets, and mitigates the impact of remaining malicious edges by adding massive important edges. An interesting observation is that the LLM fine-tuned on TAPE-Arxiv performs best at identifying malicious edges on OGBN-Arxiv, which indicates that using domain-specific LLMs can further enhance LLM4RGNN.

6 Related Work

6.1 Adversarial Attack and Defense on Graph

Extensive studies [19, 25, 32, 43, 51, 52] have shown that attackers can catastrophically degrade the performance of GNNs by maliciously perturbing the graph structure. For example, Nettack [61] first to study the adversarial attacks on graph data, which preserves degree distribution and imposes constraints on feature co-occurrence to generate small deliberate perturbations. Subsequently, Mettack [62] utilizes meta-learning while Minmax and PGD [45] utilize projected gradient descent, to solve the bilevel problem underlying poisoning attacks. Threatened by adversarial attacks, many methods [8, 9, 16, 23] have been proposed to defend against adversarial attacks. They are mainly categorized into model-centric and data-centric. The methods of model-centric improve the robustness through model enhancement, either by robust training schemes (e.g., adversarial training [8, 23]) or designing new model architectures (e.g., RGCN [60], HANG [56], Mid-GCN[16]). The methods of data-centric typically focus on flexible data processing. By treating

the attacked topology as noisy, defenders primarily purify graph structures by calculating various similarities between node embeddings [6, 20, 24, 44, 53]. For instance, ProGNN [20] jointly trains GNN and learns a clean adjacency matrix with graph properties. STABLE [24] learns effective representations by unsupervised pre-training to refine graph structures.

6.2 LLMs for Graph

Recently, LLMs have been widely employed in graph-related tasks, which even outperform traditional GNN-based methods. According to the role played by LLMs in graph-related tasks, some methods utilize LLMs as an enhancer [14, 27, 46], to enhance the quality of node features. Some methods directly use LLMs as a predictor [3, 40, 50, 55], where the graph structure is described in natural language for input to LLMs for prediction. Additionally, some methods employ LLMs as an annotator [4], generator [49], and controller [39]. Notably, recent GraphEdit [11] is a data augmentation framework, which aims to improve the performance ceiling on the original graph. In comparison, LLM4RGNN is a defense framework against topology attacks, which aims to improve the stability of various GNNs under attack. Although both focus on utilizing LLMs for graph structure learning, their motivations are distinct.

7 Conclusion

In this paper, we first explore the potential of LLMs on the graph adversarial robustness. Specifically, we propose a novel LLM-based robust graph structure inference framework, LLM4RGNN, which distills the inference capability of GPT-4 into a local LLM for identifying malicious edges and an LM-based edge predictor for finding missing important edges, to efficiently purify attacked graph structure, making GNNs more robust. Extensive experiments demonstrate that LLM4RGNN significantly improves various GNNs' robustness. Considering there are some graphs that lack textual information, a plan is to extend LLM4RGNN to graphs without text.

Acknowledgments

This work is supported by the National Key Research and Development Program of China (2023YFF0725103), the National Natural Science Foundation of China (U22B2038, 62322203, 62172052, 62192784), and the Young Elite Scientists Sponsorship Program (No.2023QNRC001) by CAST.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Yupeng Chang, Xu Wang, Jindong Wang, et al. A survey on evaluation of large language models. *ACM Transactions on Intelligent Systems and Technology*, 2024.
- [3] Zhikai Chen, Haitao Mao, Hang Li, et al. Exploring the potential of large language models (llms) in learning on graphs. *ACM SIGKDD Explorations Newsletter*, 2024.
- [4] Zhikai Chen, Haitao Mao, Hongzhi Wen, et al. Label-free node classification on graphs with large language models (llms). In *The Twelfth International Conference on Learning Representations*, 2023.
- [5] Jinlong Du, Senzhang Wang, Hao Miao, et al. Multi-channel pooling graph neural networks. In *IJCAI*, 2021.
- [6] Negin Entezari, Saba A Al-Sayouri, Amirali Darvishzadeh, et al. All you need is low (rank) defending against adversarial attacks on graphs. In *Proceedings of the 13th international conference on web search and data mining*, 2020.
- [7] C Lee Giles, Kurt D Bollacker, and Steve Lawrence. Citeseer: An automatic citation indexing system. In *Proceedings of the third ACM conference on Digital libraries*, 1998.
- [8] Lukas Gosch, Simon Geisler, Daniel Sturm, et al. Adversarial training for graph neural networks: Pitfalls, solutions, and new directions. *Advances in Neural Information Processing Systems*, 2024.
- [9] Lukas Gosch, Daniel Sturm, Simon Geisler, et al. Revisiting robustness in graph machine learning. In *The Eleventh International Conference on Learning Representations*, 2023.
- [10] Yuxin Guo, Deyu Bo, Cheng Yang, Zhiyuan Lu, Zhongjian Zhang, Jixi Liu, Yufei Peng, and Chuan Shi. Data-centric graph learning: A survey. *IEEE Transactions on Big Data*, 2024.
- [11] Zirui Guo, Lianghao Xia, Yanhua Yu, et al. Graphedit: Large language models for graph structure learning. *arXiv preprint arXiv:2402.15183*, 2024.
- [12] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 2017.
- [13] Zellig S Harris. Distributional structure. *Word*, 1954.
- [14] Xiaoxin He, Xavier Bresson, Thomas Laurent, et al. Harnessing explanations: Llm-to-llm interpreter for enhanced text-attributed graph representation learning. In *The Twelfth International Conference on Learning Representations*, 2023.
- [15] Weihua Hu, Matthias Fey, Marinka Zitnik, et al. Open graph benchmark: Datasets for machine learning on graphs. *Advances in neural information processing systems*, 2020.
- [16] Jincheng Huang, Lun Du, Xu Chen, et al. Robust mid-pass filtering graph convolutional networks. In *Proceedings of the ACM Web Conference 2023*, 2023.
- [17] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- [18] Wei Jin, Tyler Derr, Yiqi Wang, et al. Node similarity preserving graph convolutional networks. In *Proceedings of the 14th ACM international conference on web search and data mining*, 2021.
- [19] Wei Jin, Yaxing Li, Han Xu, et al. Adversarial attacks and defenses on graphs. *ACM SIGKDD Explorations Newsletter*, 2021.
- [20] Wei Jin, Yao Ma, Xiaorui Liu, et al. Graph structure learning for robust graph neural networks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020.
- [21] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.
- [22] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.
- [23] Kuan Li, YiWen Chen, Yang Liu, et al. Boosting the adversarial robustness of graph neural networks: An ood perspective. In *The Twelfth International Conference on Learning Representations*, 2023.
- [24] Kuan Li, Yang Liu, Xiang Ao, et al. Reliable representations make a stronger defender: Unsupervised structure refinement for robust gnn. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2022.
- [25] Kuan Li, Yang Liu, Xiang Ao, et al. Revisiting graph adversarial attack and defense from a data distribution perspective. In *The Eleventh International Conference on Learning Representations*, 2022.
- [26] Yaxin Li, Wei Jin, Han Xu, and Jiliang Tang. Deeprobust: A pytorch library for adversarial attacks and defenses. *arXiv preprint arXiv:2005.06149*, 2020.
- [27] Hao Liu, Jiarui Feng, Lecheng Kong, et al. One for all: Towards training one graph model for all classification tasks. In *The Twelfth International Conference on Learning Representations*, 2023.
- [28] Jiawei Liu, Cheng Yang, Zhiyuan Lu, et al. Towards graph foundation models: A survey and beyond. *arXiv preprint arXiv:2310.11829*, 2023.
- [29] Chengsheng Mao, Liang Yao, and Yuan Luo. Medgcn: Graph convolutional networks for multiple medical tasks. *arXiv preprint arXiv:1904.00326*, 2019.
- [30] Andrew Kachites McCallum, Kamal Nigam, et al. Automating the construction of internet portals with machine learning. *Information Retrieval*, 2000.
- [31] Tomas Mikolov, Ilya Sutskever, Kai Chen, et al. Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 2013.
- [32] Felix Mujkanovic, Simon Geisler, et al. Are defenses for graph neural networks robust? *Advances in Neural Information Processing Systems*, 2022.
- [33] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.
- [34] Stephen Robertson. Understanding inverse document frequency: on theoretical arguments for idf. *Journal of documentation*, 2004.
- [35] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, et al. Collective classification in network data. *AI magazine*, 2008.
- [36] Lichao Sun, Yingdong Dou, et al. Adversarial attack and defense on graph data: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 2022.
- [37] Hugo Touvron, Thibaut Lavril, Gautier Izacard, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [38] Petar Veličković, Guillem Cucurull, Arantxa Casanova, et al. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.
- [39] Haishuai Wang, Yang Gao, Xin Zheng, et al. Graph neural architecture search with gpt-4. *arXiv preprint arXiv:2310.01436*, 2023.
- [40] Heng Wang, Shangbin Feng, Tianxing He, et al. Can language models solve graph problems in natural language? *Advances in Neural Information Processing Systems*, 2024.
- [41] Jianian Wang, Sheng Zhang, Yanghua Xiao, et al. A review on graph neural network methods in financial applications. *arXiv preprint arXiv:2111.15367*, 2021.
- [42] Xiao Wang, Peng Cui, Jing Wang, et al. Community preserving network embedding. In *Proceedings of the AAAI conference on artificial intelligence*, 2017.
- [43] Marcin Waniek, Tomasz P Michalak, Michael J Wooldridge, et al. Hiding individuals and communities in a social network. *Nature Human Behaviour*, 2018.
- [44] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, et al. Adversarial examples for graph data: deep insights into attack and defense. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, 2019.
- [45] Kaidi Xu, Hongge Chen, Sijia Liu, et al. Topology attack and defense for graph neural networks: An optimization perspective. In *International Joint Conference on Artificial Intelligence*, 2019.
- [46] Ronghui Xu, Hao Miao, et al. Pefad: A parameter-efficient federated framework for time series anomaly detection. In *SIGKDD*, 2024.
- [47] Xiaohan Xu, Ming Li, Chongyang Tao, et al. A survey on knowledge distillation of large language models. *arXiv preprint arXiv:2402.13116*, 2024.
- [48] Shukang Yin, Chaoyou Fu, Sirui Zhao, et al. A survey on multimodal large language models. *arXiv preprint arXiv:2306.13549*, 2023.
- [49] Jianxiang Yu, Yuxiang Ren, Chenghua Gong, et al. Empower text-attributed graphs learning with large language models (llms). *arXiv preprint arXiv:2310.09872*, 2023.
- [50] Mengmei Zhang, Mingwei Sun, Peng Wang, et al. Graphtranslator: Aligning graph model to large language model for open-ended tasks. In *Proceedings of the ACM on Web Conference 2024*, 2024.
- [51] Mengmei Zhang, Xiao Wang, Chuan Shi, et al. Minimum topology attacks for graph neural networks. In *Proceedings of the ACM Web Conference 2023*, 2023.
- [52] Mengmei Zhang, Xiao Wang, Meiqi Zhu, et al. Robust heterogeneous graph neural networks against adversarial attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022.
- [53] Xiang Zhang and Marinka Zitnik. Gnn-guard: Defending graph neural networks against adversarial attacks. *Advances in neural information processing systems*, 2020.
- [54] Zhongjian Zhang, Mengmei Zhang, Yue Yu, et al. Endowing pre-trained graph models with provable fairness. In *Proceedings of the ACM on Web Conference 2024*, 2024.
- [55] Haiteng Zhao, Shengchao Liu, Ma Chang, et al. Gimlet: A unified graph-text model for instruction-based molecule zero-shot learning. *Advances in Neural Information Processing Systems*, 2024.
- [56] Kai Zhao, Qiyu Kang, Yang Song, et al. Adversarial robustness in graph neural networks: A hamiltonian approach. *Advances in Neural Information Processing Systems*, 2024.
- [57] Wayne Xin Zhao, Kun Zhou, Junyi Li, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
- [58] Qinkai Zheng, Xu Zou, Yuxiao Dong, et al. Graph robustness benchmark: Benchmarking the adversarial robustness of graph machine learning. *Neural Information Processing Systems Track on Datasets and Benchmarks 2021*, 2021.
- [59] Huichi Zhou, Zhaoyang Wang, Hongtao Wang, et al. Evaluating the validity of word-level adversarial attacks with large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, 2024.
- [60] Dingyuan Zhu, Ziwei Zhang, Peng Cui, et al. Robust graph convolutional networks against adversarial attacks. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*, 2019.
- [61] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. Adversarial attacks on neural networks for graph data. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*, 2018.
- [62] Daniel Zügner and Stephan Günnemann. Adversarial attacks on graph neural networks via meta learning. In *International Conference on Learning Representations (ICLR)*, 2019.

Table 11: Dataset statistics.

Dataset	#Nodes	#Edges	#Classes	#Features	Method
Cora	2,708	5,429	7	1,433	BoW
Citeseer	3,186	4,225	6	3,113	BoW
PubMed	19,717	44,338	3	500	TF-IDF
OGBN-Arxiv (subset)	14,167	33,520	40	128	skip-gram
OGBN-Products (subset)	12,394	29,676	47	100	BoW
TAPE-Arxiv23	46,198	78,548	40	300	word2vec

A Experiment Detail

A.1 Dataset

In this paper, we select the TAPE-Arxiv23 [14] with up-to-date and rich texts to construct the instruction dataset, and use the five popular node classification datasets to evaluate model performance: Cora [30], Citeseer [7], Pubmed [35], OGBN-Arxiv [15] and OGBN-Products [15]. For each graph, following existing works [20, 24], we randomly split the nodes into 10% for training, 10% for validation, and 80% for testing. Notably, for OGBN-Arxiv (with 169,343 nodes and 1,166,243 edges) and OGBN-Products (with 2 million nodes and 61 million edges), in the main experiments, since we consider the strongest Mettack and Nettack, with high complexity, constructing their attack topologies is not feasible. Thus, we adopt a node sampling strategy [12] to obtain their subgraph. We give detailed statistics of each dataset in Table 11.

A.2 Baseline

- **GCN**: GCN is a popular graph convolutional network based on spectral theory.
- **GAT**: GAT is composed of multiple attention layers, which can learn different weights for different neighborhood nodes. It is often used as a baseline for defending against adversarial attacks.
- **RGCN**: RGCN models node representations as Gaussian distributions to mitigate the impact of adversarial attacks, and employs an attention mechanism to penalize nodes with high variance.
- **Simp-GCN**: SimpGCN employs a k NN graph to maintain the proximity of nodes with similar features in the representation space and uses self-learned regularization to preserve the remoteness of nodes with differing features.
- **ProGNN**: ProGNN adapts three regularizations of graphs, i.e., feature smoothness, low-rank, and sparsity, and learns a clean adjacency matrix to defend against adversarial attacks.
- **STABLE**: STABLE is a pre-training model, which specifically learns effective representations to refine graph structures.
- **HANG-quad**: HANG-quad incorporates conservative Hamiltonian flows with Lyapunov stability to various GNNs, to improve their robustness against adversarial attacks.
- **GraphEdit**: GraphEdit utilizes LLMs to identify noisy connections and uncover implicit relations among non-connected nodes in the original graph. Notably, GraphEdit only provides prompts for Cora, Citeseer and Pubmed datasets.

A.3 Implementation Detail

For LLM4RGNN, we select Mistral-7B as our local LLM. Based on GPT-4, we construct approximately 26,518 instances for tuning LLMs and use the LoRA method to achieve parameter-efficient fine-tuning. To address the potential problem of label imbalance in training LM-based edge predictor, we select the 4,000 node pairs

Table 12: The token cost of different instruction datasets.

Size	Input Token	Output Token	Cost
5,000	2,988,658	561,762	\$46.7
15,000	8,951,923	1,681,254	\$139.9
26,518	15,809,910	2,970,500	\$247.2

with the lowest cosine similarity to construct the candidate set. We set the hyper-parameters of LLM4RGNN as follows: For local LLMs, when no purification occurs, the purification threshold β is selected from $\{1, 2\}$ to prevent deleting too many edges; otherwise, it is selected from $\{2, 3, 4\}$. For LM-based edge predictor, the threshold γ is tuned from $\{0.91, 0.93, 0.95, 0.97, 0.99\}$ and the number of edges K is tuned from $\{1, 3, 5, 7, 9\}$.

We use DeepRobust[26], an adversarial attack repository, to implement all attacks as well as GCN, GAT, RGCN, and Sim-PGCN. We implement ProGNN, STABLE, and HANG-quad with the code provided by the authors. To facilitate fair comparisons, we tune all baselines' parameters using a grid search strategy. Unless otherwise specified, we adopt the default parameter setting in the author's implementation. For GCN, the hidden size is 256 for OGBN-Arxiv and 16 for others. For GAT, the hidden size is 128 for OGBN-Arxiv and 8 for others. For RGCN and Sim-PGCN, the hidden size is 256 for OGBN-Arxiv and 128 for others. For Sim-PGCN, we tune the weighting parameter λ is searched from $\{0.1, 0.5, 1, 5, 10, 50, 100\}$ and γ is searched from $\{0.01, 0.1\}$. For STABLE, we tune the Jac-card similarity threshold t_1 from $\{0.0, 0.01, 0.02, 0.03, 0.04, 0.05\}$, k is tuned from $\{1, 3, 5, 7, 11, 13\}$, α is tuned from -0.5 to 3 . For HANG-quad, we tune the time from $\{3, 6, 8, 15, 20, 25\}$, the hidden size from $\{16, 32, 64, 128\}$, and dropout from $\{0.2, 0.4, 0.6, 0.8\}$. For all experiments, we select the optimal hyper-parameters on the validation set and apply them to the test set.

A.4 Computing Environment and Resource

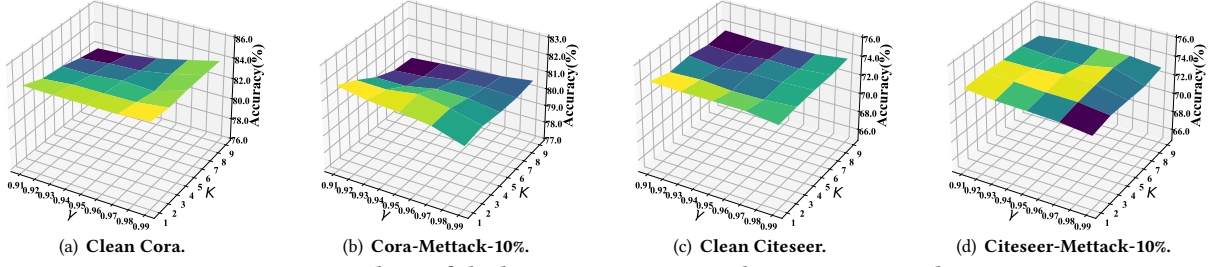
The implementation of LLM4RGNN utilized the PyG module. With GPT-4, the token costs for constructing different sizes of instruction datasets are reported in Table 12, and it is "spend once, use forever". The minimum resource requirements for LLMs are as follows: The 7B, 8B and 13B LLMs require 16G, 20G, and 32G for fine-tuning, and 15G, 16G, and 26G for deploying. The experiments are conducted in a computing environment with the following specifications:

- **OS**: Linux ubuntu 5.15.0-102-generic.
- **CPU**: Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz.
- **GPU**: NVIDIA A800 80GB.

B More Experiment Result

B.1 Defense against Inductive Poisoning Attack

We further evaluate the generalization of LLM4RGNN under inductive poisoning attacks. We conduct inductive experiments with Mettack on the Cora and Citeseer. Specifically, we randomly split the data into training, validation, and test sets with a 1:8:1 ratio. During training, we ensure the removal of test nodes and their connected edges from the graph. We perform Mettack on the validation set, purify the attacked graph using LLM4RGNN, and use the purified graph to train GNNs. The trained GNNs are then predicted on the clean test set. As reported in Table 13, where we only report the baselines that support the inductive setting, results show

Figure 9: Analysis of the hyper-parameter γ and K against Mettack.Table 13: Node classification accuracy ($\% \pm \sigma$) under Inductive Mettack. The best results are in bold.

Dataset	Ptb Rate	HANG-quad	GCN		GAT	
			Vanilla	LLM4RGNN	Vanilla	LLM4RGNN
Cora	0%	82.09 $\pm$ 0.40	84.36 $\pm$ 0.28	84.25 $\pm$ 0.16	84.35 $\pm$ 0.62	84.39 $\pm$ 0.61
	5%	75.85 $\pm$ 0.67	77.35 $\pm$ 1.56	83.17 $\pm$ 0.17	81.20 $\pm$ 0.93	83.68 $\pm$ 0.26
	10%	70.95 $\pm$ 1.26	72.79 $\pm$ 1.49	83.02 $\pm$ 0.34	78.55 $\pm$ 1.12	83.48 $\pm$ 0.49
	20%	58.81 $\pm$ 1.99	61.56 $\pm$ 2.64	82.69 $\pm$ 0.65	67.65 $\pm$ 2.69	83.14 $\pm$ 0.69
Citeseer	0%	73.15 $\pm$ 0.94	73.76 $\pm$ 0.34	74.07 $\pm$ 0.51	73.84 $\pm$ 0.57	74.26 $\pm$ 0.69
	5%	71.50 $\pm$ 0.55	70.60 $\pm$ 0.56	73.35 $\pm$ 0.35	72.47 $\pm$ 0.82	73.25 $\pm$ 0.91
	10%	69.42 $\pm$ 0.64	67.64 $\pm$ 0.73	72.68 $\pm$ 0.41	71.11 $\pm$ 1.02	73.25 $\pm$ 0.40
	20%	67.90 $\pm$ 1.21	61.35 $\pm$ 1.35	72.54 $\pm$ 0.36	65.76 $\pm$ 2.85	73.32 $\pm$ 0.52

that under the inductive setting, LLM4RGNN not only consistently improves the robustness of various GNNs but also surpasses the robust GCN framework HANG-quad, demonstrating its superior defensive capability in the inductive setting.

Table 14: The performance of LLM4RGNN with different prompt variants. The best results are in bold.

Method	Cora		Citeseer	
	ACC ($\uparrow$)	AdvEdge ($\downarrow$)	ACC ($\uparrow$)	AdvEdge ($\downarrow$)
Filter	81.02$\pm$1.00	114(2.16%)	73.06$\pm$0.69	108(2.56%)
Auxiliary Label	80.80 $\pm$ 1.07	165(3.13%)	72.17 $\pm$ 0.75	121(2.86%)

B.2 The Impact of Different Prompt Variant

Recall that we utilize the ground truth of the query edge set to filter the incorrect prediction of GPT-4, there is another alternative is to directly take the ground truth as input of GPT-4. Specifically, we require GPT-4 to provide analysis and edge ratings ranging from 4 to 6 for a non-malicious edge, while providing edge ratings ranging from 1 to 3 for a malicious edge. Considering an instruction dataset with a size of 7500, we compared the results of the aforementioned method (Auxiliary Label) with our method (Filter). As reported in Table 14, the performance variations of LLM4RGNN are minimal, demonstrating that LLM4RGNN is not sensitive to prompt design. Moreover, the reason why the lower performance of Auxiliary Label is that the edge label may be considered by GPT-4 during the generating analysis, but it is inaccessible during testing.

Table 15: Performance comparison between Mistral-7B and Llama3-8B under Mettack. The best results are in bold.

Dataset	Ptb Rate	AdvEdge ($\downarrow$)		ACC ($\uparrow$) w/o EP		ACC ($\uparrow$) Full	
		Mistral-7B	Llama3-8B	Mistral-7B	Llama3-8B	Mistral-7B	Llama3-8B
Cora	0%	-	-	83.78 $\pm$ 0.38	83.72 $\pm$ 0.30	84.13 $\pm$ 0.33	84.34$\pm$0.56
	5%	28(0.53%)	30(0.57%)	80.48 $\pm$ 0.19	80.60$\pm$0.27	81.76 $\pm$ 0.69	81.98$\pm$0.55
	10%	60(1.14%)	63(1.19%)	79.83 $\pm$ 0.43	79.96$\pm$0.39	81.80 $\pm$ 0.76	81.77 $\pm$ 0.44
	20%	102(1.93%)	107(2.03%)	78.68 $\pm$ 0.28	79.22$\pm$0.34	81.41 $\pm$ 0.77	81.71$\pm$0.51
Citeseer	0%	-	-	73.39 $\pm$ 0.69	73.41 $\pm$ 0.65	74.20 $\pm$ 0.56	73.84 $\pm$ 0.50
	5%	30(0.71%)	25(0.59%)	72.17 $\pm$ 0.57	72.44$\pm$0.52	73.94 $\pm$ 0.56	73.28 $\pm$ 1.19
	10%	55(1.30%)	47(1.11%)	71.50 $\pm$ 0.59	71.32 $\pm$ 0.58	73.62 $\pm$ 0.39	73.20 $\pm$ 0.68
	20%	92(2.18%)	91(2.15%)	71.07 $\pm$ 0.35	70.85 $\pm$ 0.64	74.12 $\pm$ 0.85	74.06 $\pm$ 0.78

B.3 More Hyper-parameter Sensitivity

We conduct more hyper-parameter experiments on the probability threshold λ and the number of important edges K . As shown in Figure 9, results indicate that the performance of the LLM4RGNN is stable under various parameter configurations and consistently outperforms existing SOTA methods.

B.4 Comparison of Llama3-8B vs. Mistral-7B

In Section 5.3.3, Llama3-8B and Mistral-7B achieved close performance. Given that Llama3-8B is released after all experiments are completed, we conduct a comprehensive comparison between them here. As reported in Table 15, both well-tuned LLMs can effectively identify malicious edges, and the performance gap is negligible.

C Case Study

Here, we show some cases using GPT-4 and well-tuned Mistral-7B to infer the relationships between nodes. We find that the well-tuned Mistral-7B possesses the edge relation inference ability of GPT-4. They infer the edge relations and provide analysis by discussing the background, problems, methods, and applications of the two nodes.

C.1 TAPE-Arxiv23 (GPT-4)

Analysis: The two nodes represent research in different domains: Node1 discusses renewable energy and building thermal mass in the context of smart grids, while Node2 focuses on hiding sound files in images using k-LSB mutation and Cuckoo Search. There is no direct connection between the optimization of renewable energy in buildings and the embedding of sound files in images. **The topics are unrelated, and the research methods and applications are distinct.**
Relevance Score: 1

C.2 Cora (Mistral-7B)

Analysis: The two nodes represent research in different domains: Node1 discusses a neural network model of memory consolidation in the brain, while Node2 focuses on a software tool for generating 3D forms in architecture using evolutionary algorithms. There is no direct connection between the brain's memory consolidation process and the evolution of 3D forms in architecture. **The topics are unrelated, and the expertise required for each is distinct.**
Relevance Score: 1